Week-1

Backend Engineering Launchpad

Aashray Agarwal

Airtribe

Backend Development Fundamentals of Java

Airtribe

Meet your Instructor

Aashray, a talented Software Engineer at Uber, brings rich experience from his previous roles at Google and Amazon. As a dedicated Backend Engineer, he holds a strong interest in Distributed Systems, System Design, Data Structures, Algorithms, and Competitive Programming. Proficient in Java, C, and C++, Aashray combines technical excellence with a collaborative and disciplined approach, making him a valuable mentor for aspiring engineers.

Aashray Agarwal

Software engineer @Uber Uber

Airtribe

Agenda

- Introduction to Java
- Java Features
- Fundamentals of Java
- Basic syntax and control structures
- Why use java ?
- Exception handling in java
- Live coding

What is java

- Java is a high-level, class-based, object-oriented programming language designed to have as few implementation dependencies as possible.
- Created by Sun Microsystems, now owned by Oracle.
- Known for its "write once, run anywhere" capability due to its platform-independent bytecode.

Key features of java

- Simple
- Object Oriented
- Platform Independent
- Robust and secure
- Multithreaded
- High performance
- Distributed

Why Java

- Platform Independence
- Scalability and Performance
- Enterprise-Ready
- Community Support

Where to use Java ?

- CPU intensive applications

Java Fundamentals

- **JDK (Java Development Kit)**:

  Includes tools for developing Java applications.

  Compiler (javac), Javadoc, Java debugger.

- **JRE (Java Runtime Environment)**:

  Provides the runtime environment for Java applications.

  Includes JVM, core libraries, and runtime environment to execute Java bytecode.

- **JVM (Java Virtual Machine)**:

  Executes Java bytecode.

  Provides platform independence by running bytecode on different platforms.

Java Fundamentals

- **Bytecode**:

  Intermediate representation of Java source code.

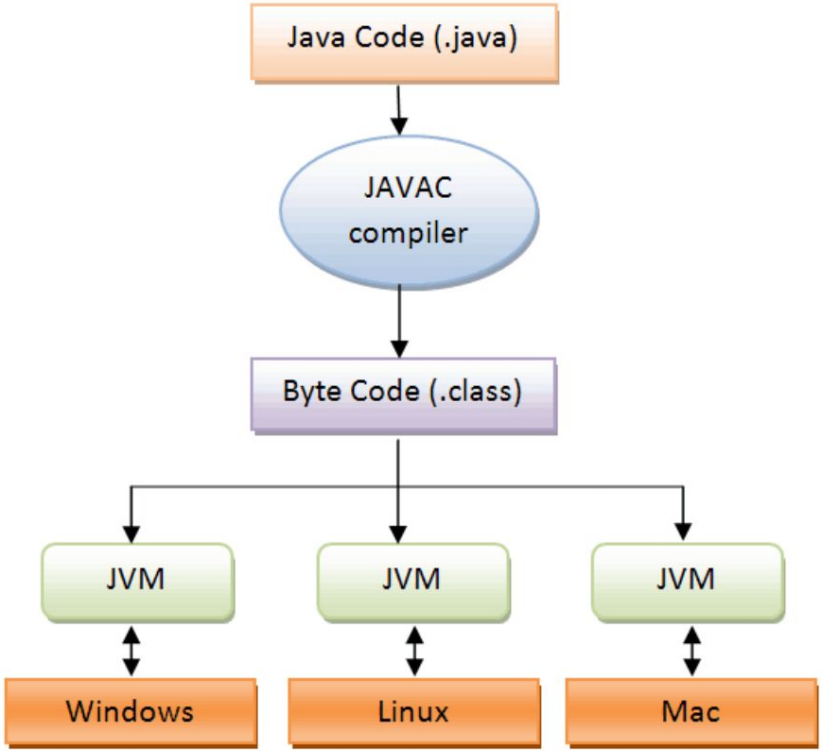
  Generated by the Java compiler (javac).

- **Platform Independence**:

  Java bytecode is platform-independent.

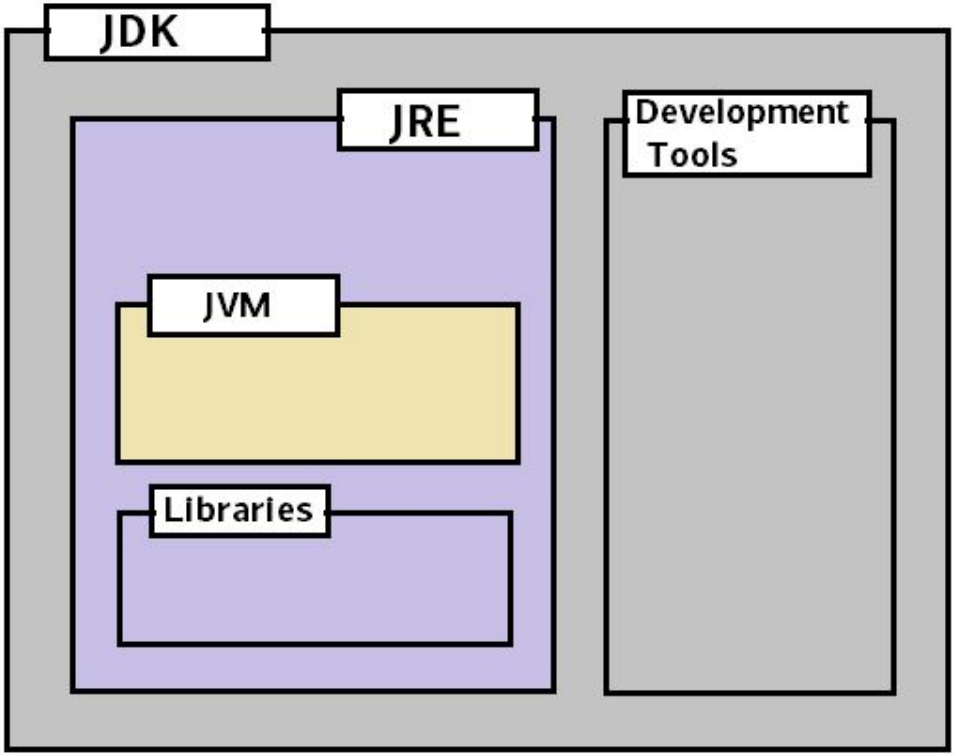
  It can be executed on any system with a compatible JVM.

Platform Independence



JDK Components

JDK Components



How does JVM work ?

Source code
(Main.java)

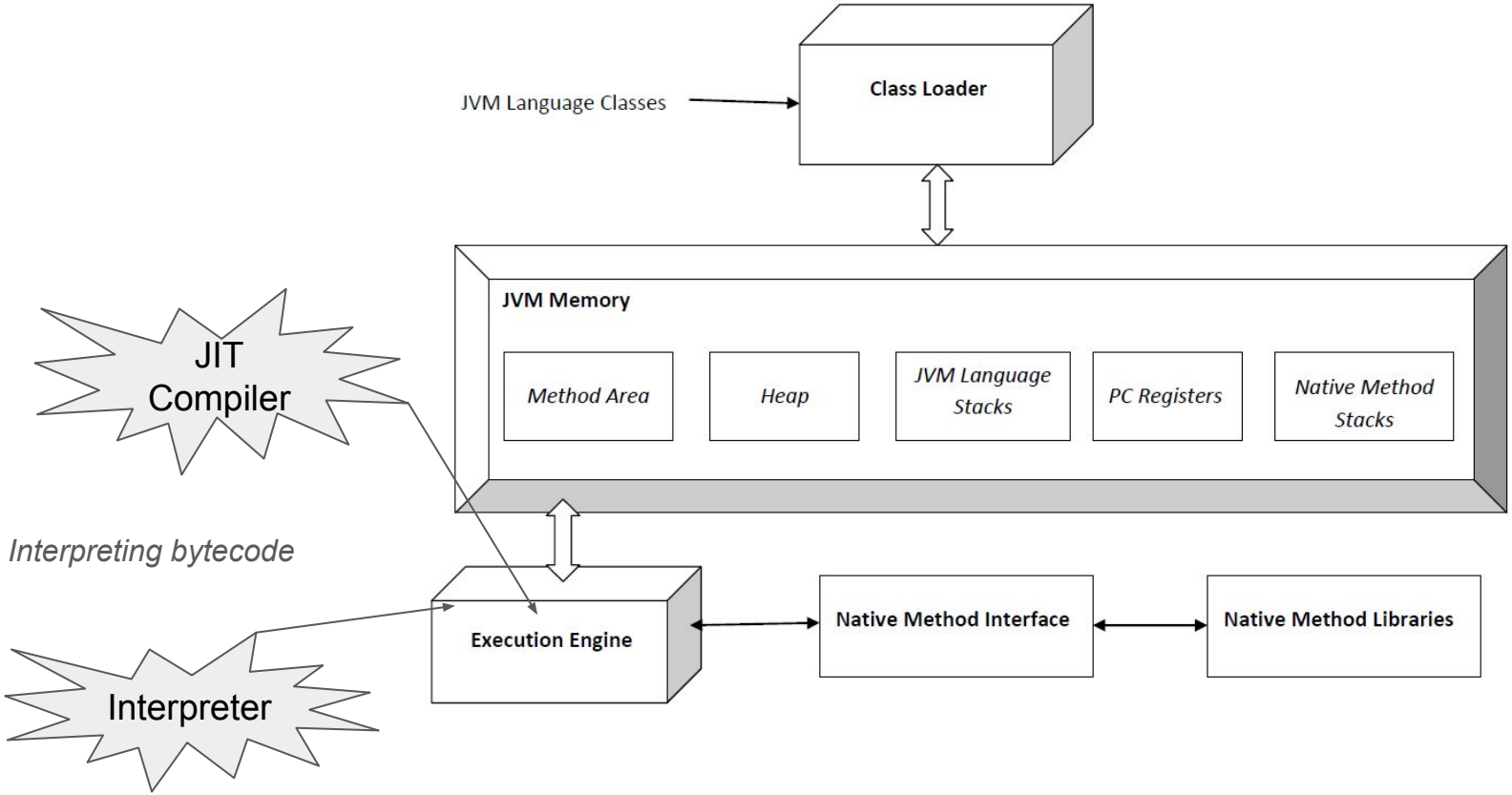
Compiler
(javac)

Byte code
(Main.class)

JVM

Program
Execution

JVM Components



Airtribe

JVM Components – 100 feet view

Class Loader Subsystem

- Loads .class files into memory.
- Performs **Loading, Linking, and Initialization**.
- Ensures **bytecode verification** before execution.

Runtime Memory Areas (JVM Memory)

Manages memory for class metadata, objects, and execution. It includes:

- **Method Area** → Stores class metadata, static variables, method code.
- **Heap** → Stores objects and instance variables.
- **Java Stack** → Stores method call frames, local variables.
- **PC Register** → Keeps track of the current instruction being executed.
- **Native Method Stack** → Manages native (JNI) method execution.

JVM Components – 100 feet view

Execution Engine

The core of the JVM that runs the program:

- **Interpreter** → Executes bytecode line-by-line (slow but startup-friendly).
- **Just-In-Time (JIT) Compiler** → Converts bytecode to **native machine code** for better performance.
- **Garbage Collector (GC)** → Automatically frees up memory by removing unused objects.

Native Interface (JNI - Java Native Interface)

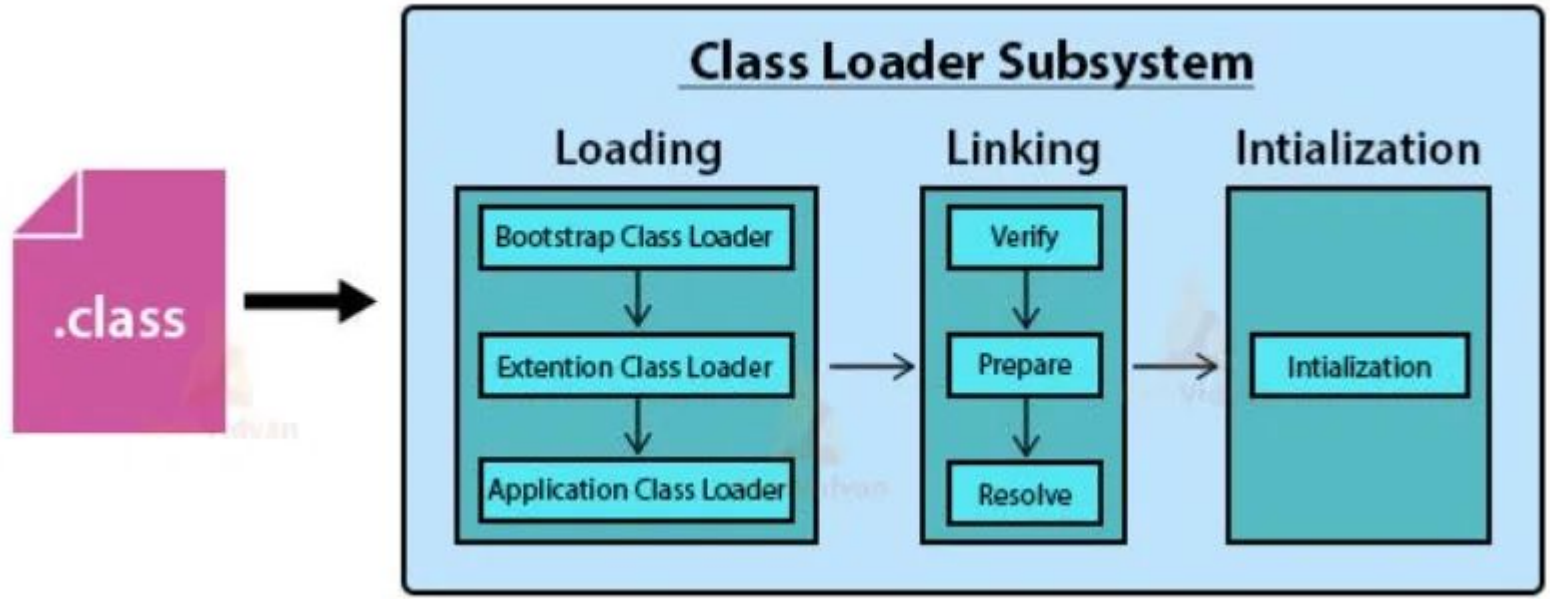
- Enables Java code to call native code written in **C, C++**, etc.
- Used for system-level operations and performance optimizations.

JVM Components – 100 feet view

Native Libraries & Java Native Libraries (JVM Support)

- Loads essential system libraries required for JVM execution.
- Supports OS-specific interactions via native code.

Class Loader Subsystem



Different JVM Class Loaders

Bootstrap ClassLoader (Primordial ClassLoader)

- **Loads**: Core Java classes (rt.jar in Java 8, java.base module in Java 9+)
- **Example**: java.lang.String, java.util.List
- **Parent**: **None** (it's part of the JVM itself, written in C++)
- **Returns null when queried**
- **Code Example:**
  System.out.println(String.class.getClassLoader()); // Output: null

Different JVM Class Loaders

Extension (Platform) ClassLoader

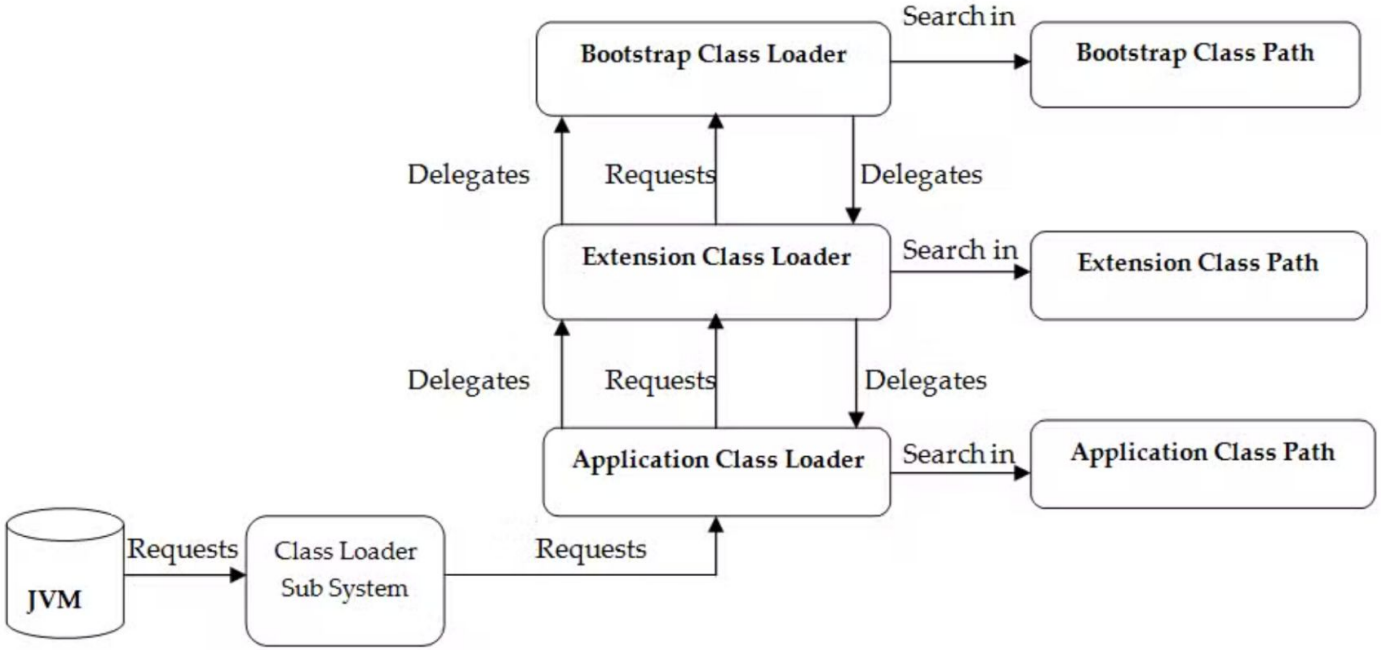
- **Loads**: Classes from lib/ext/ (Java 8) or java.ext.dirs (Removed in Java 9+)
- **Example**: com.sun.nio.zipfs.ZipFileSystemProvider.class
- **Parent**: **Bootstrap ClassLoader**
- **Java 9+ Equivalent**: **Platform ClassLoader**
- **Code Example:**
  System.out.println(ClassLoader.getSystemClassLoader().getParent());

Different JVM Class Loaders

Application (System) ClassLoader

- **Loads**: Classes from the **classpath** (-cp or CLASSPATH env variable)
- **Example**: Custom classes (com.example.MyClass)
- **Parent**: **Extension (Platform) ClassLoader**
- **Code Example:**
  System.out.println(LoaderExample.class.getClassLoader());// Output: sun.misc.Launcher$AppClassLoader@..

Class Loading Process

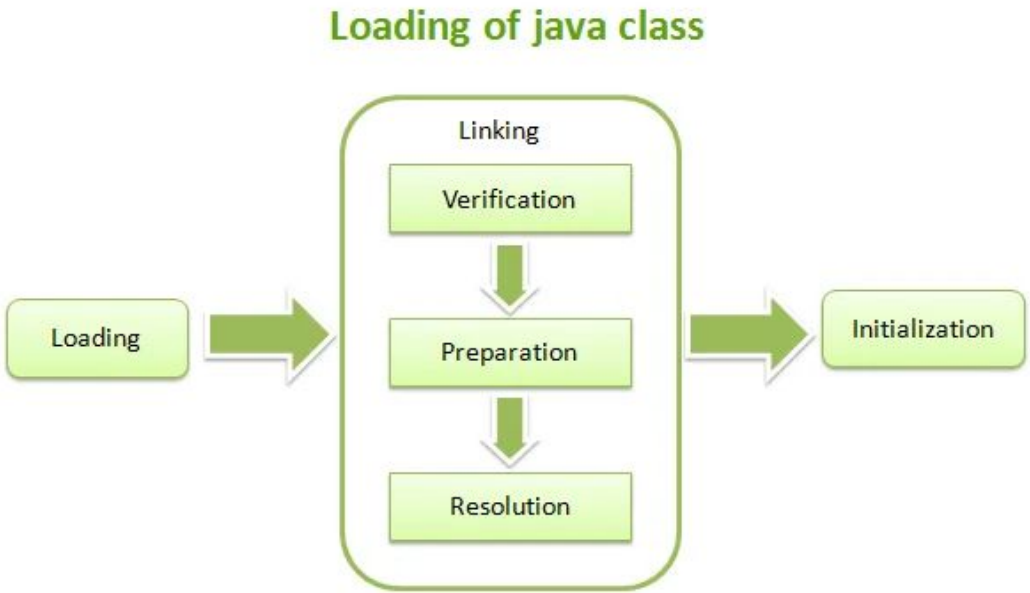


Class Loading Process – Parent Delegation Model

Whenever a class needs to be loaded:

- The **Application ClassLoader** first asks **Platform ClassLoader**.
- The **Platform ClassLoader** asks **Bootstrap ClassLoader**.
- If **Bootstrap ClassLoader** can't find it, control returns to **Platform ClassLoader**.
- If **Platform ClassLoader** can't find it, **Application ClassLoader** attempts to load it.

Linking Phase



Linking Phase

The **Linking Phase** in Java occurs after **Loading** and before **Initialization**. It ensures that the class is properly prepared for execution by verifying, preparing, and resolving its components.

The **Linking Phase** consists of three subphases:

- **Verification** – Ensures the class file is valid.
- **Preparation** – Allocates memory for static variables and assigns default values.
- **Resolution** – Converts symbolic references into actual memory addresses.

Linking – Verification, Preparation, Resolution

Verification

- Ensures the class file is correctly formatted and does not contain any illegal operations.
- Prevents security risks (e.g., memory corruption, unauthorized access).
- Uses the **Bytecode Verifier** to check the correctness of the .class file.

Preparation

- This phase **allocates memory for static variables** and assigns **default values** (not user-defined values).
- The **Method Area** is updated with memory for class-level variables.

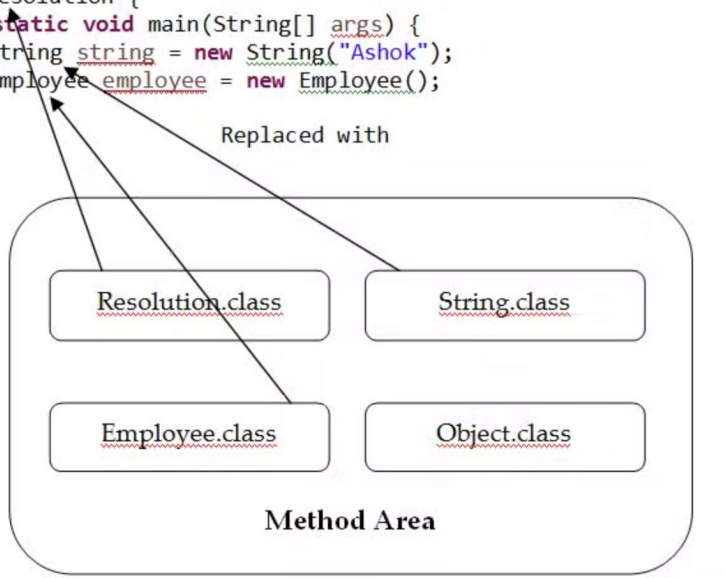
Resolution

- Converts **symbolic references** in the .class file to **actual memory addresses** (runtime references).

Resolution

```java
package com.ashok.jvm;

public class Resolution {
	public static void main(String[] args) {
		String string = new String("Ashok");
		Employee employee = new Employee();
	}
}
```



Airtribe

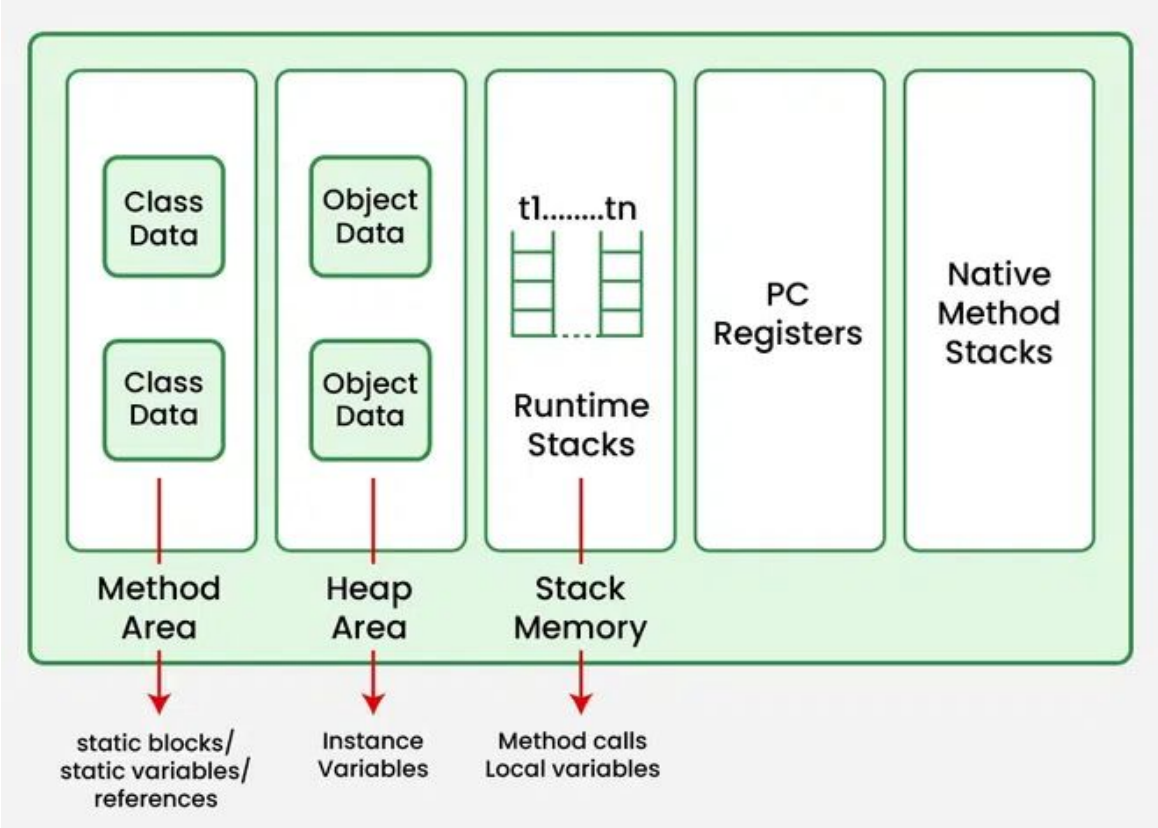
Initialization Phase

- The **Initialization Phase** is the final step of class loading in JVM. This phase ensures that all **static variables** and **static blocks** are executed in a well-defined order before the class is used.

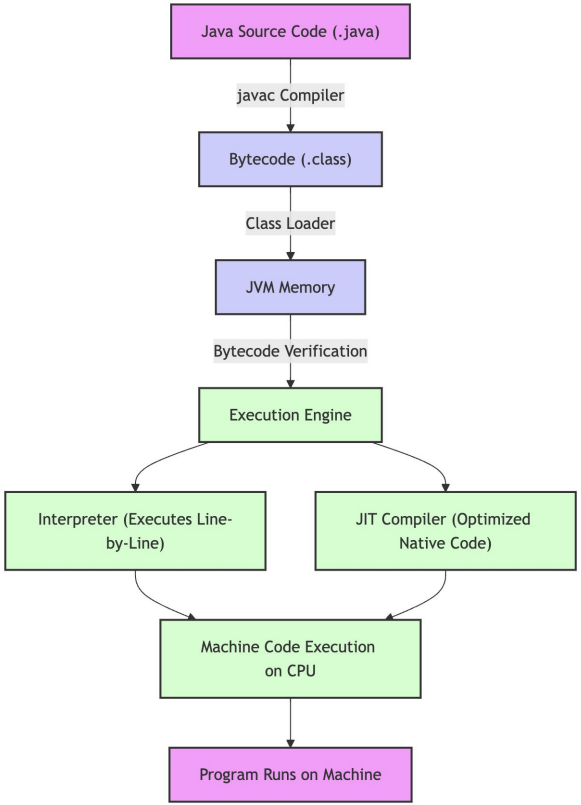
Steps of Initialization

- Execution of static variable assignments
- Execution of static blocks in the order they appear
- Ensuring superclass initialization before subclass

Runtime access area

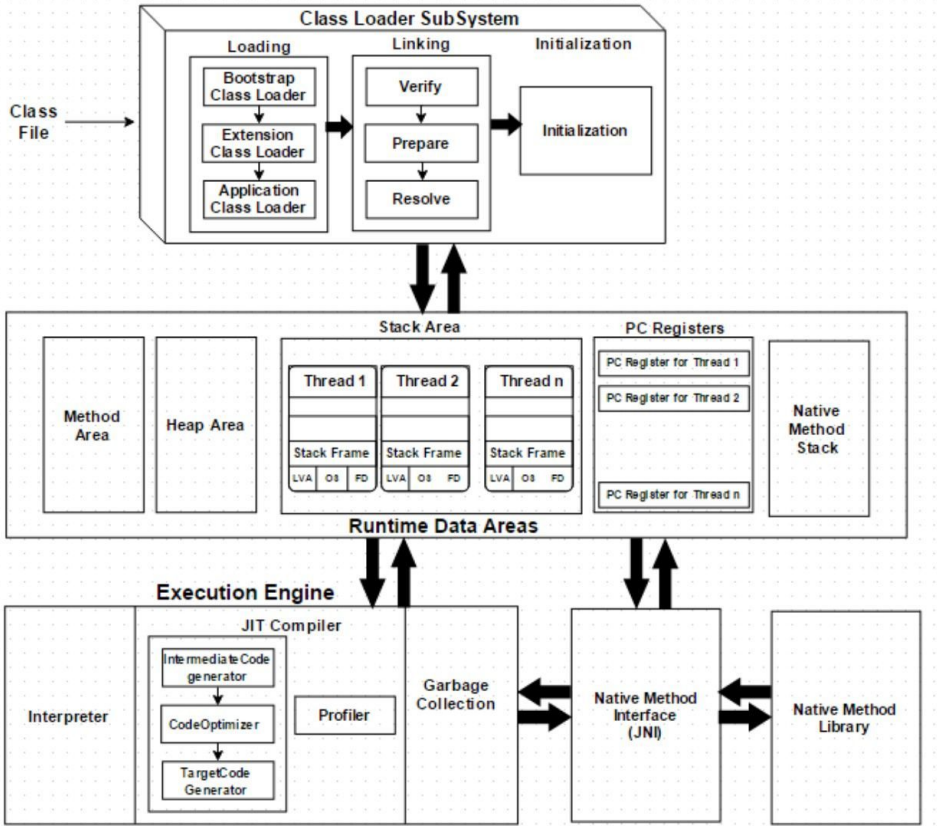


Compilation and execution process



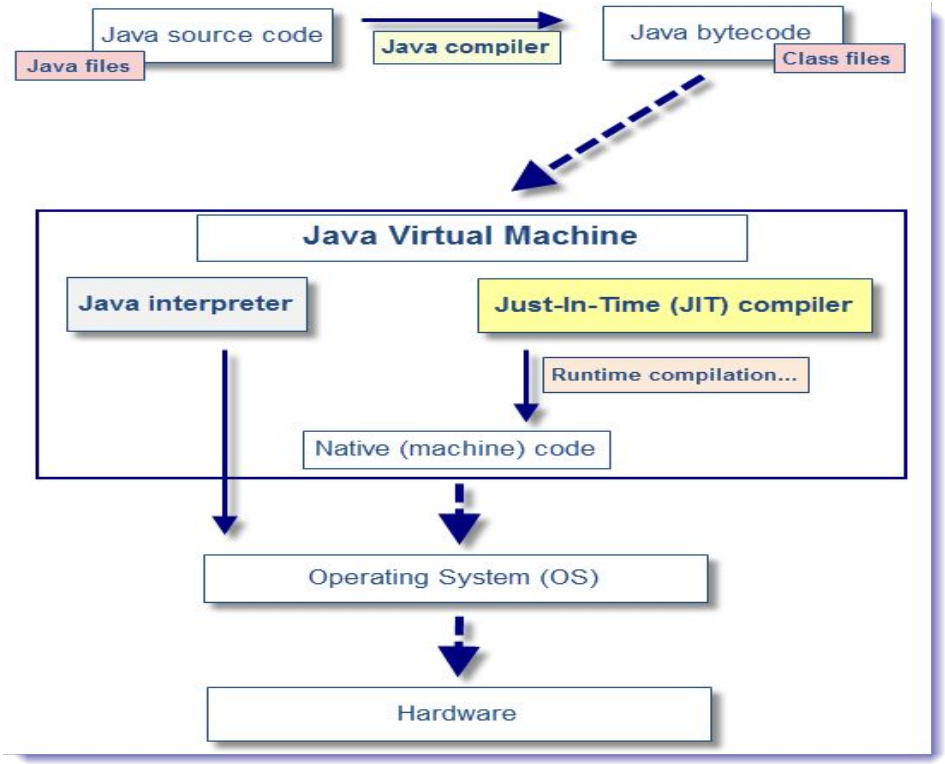
Airtribe

JVM End to End Architecture



Airtribe

Java Compilation Process



JIT vs Interpreter

- **Interpreter**:
  - **Function**: Executes Java bytecode one instruction at a time.
  - **Performance**: Slower execution due to repetitive translation.
  - **Memory Usage**: Lower memory usage; no storage of compiled code.
  - **Startup Time**: Quick, no initial compilation overhead.
  - **Use Case**: Ideal for quickly starting execution and handling rarely used code paths.
- **JIT Compiler**:
  - **Function**: Compiles Java bytecode to native machine code at runtime.
  - **Performance**: Faster execution after initial compilation; applies optimizations.
  - **Memory Usage**: Higher memory usage; stores compiled machine code.
  - **Startup Time**: Slower due to compilation overhead, but faster subsequent execution.
  - **Use Case**: Best for frequently executed code paths (hot spots) for long-term performance gains.

Compilation	Interpretation
Compilation is the process of converting the Java source code into an executable form, known as bytecode.	Interpretation is the process of executing the Java bytecode directly by the JVM.
The Java compiler takes the source code and produces bytecode, which is then executed by the Java Virtual Machine (JVM).	The bytecode is loaded into the JVM, and the JVM interprets the bytecode and executes the program.
The code cannot be compiled because of compilation errors.	Debugging is mainly done in run-time.
Compared to interpreted compiled programs operate faster.	When an interpreted program executes, changes can be made.

Compilation vs Interpretation

- Compilation converts source code into bytecode, whereas interpretation executes bytecode directly by the JVM.
- The compilation is done once during the development process, while interpretation happens each time the program is run.
- Compilation allows for the optimization of the code, while interpretation provides greater flexibility and dynamic behavior.
- Overall, both compilation and interpretation play important roles in Java programming, and the choice of which to use will depend on the specific needs of the program.

Compilation vs Interpretation

Command	Purpose
java -Xint	Runs only in interpreted mode (JIT disabled)
java -Xcomp	Runs with aggressive JIT compilation
java -XX:+PrintCompilation	Shows which methods JIT compiled
java -XX:+LogCompilation	Logs detailed JIT compilation info
java -XX:+PrintInterpreter	Prints methods executed in interpreted mode

It’s time for Breakout Rooms!

Airtribe

What is the one programming language & framework you are most comfortable with and what are its top qualities/pros & cons according to you ?

Visualizing class loading, bytecode, VM memory and other stats

- Javap -c <<Test.class>>
- Java -verbose:class <<Test.class>>
- Jps -l
- Jcmd <pid> <<Flags>>
- JvisualVM

Exception Handling

- Exceptions are events that disrupt the normal flow of program execution.

  Examples: ArithmeticException, NullPointerException, IOException, etc

- **Why Handle Exceptions?**

  Improve robustness: Prevent abrupt program termination.

  Graceful error recovery.

  Debugging aid: Understand and fix issues more effectively.

Exception hierarchy Java

- **Java's Exception Hierarchy**

  Throwable: Base class for all exceptions.

  Error: Irrecoverable issues (e.g., OutOfMemoryError).

  Exception: Recoverable issues.

  - **Checked Exceptions**: Compiler-enforced handling (e.g., IOException).
  - **Unchecked Exceptions (Runtime Exceptions)**: Runtime issues (e.g., NullPointerException).

Checked Exceptions

- **Checked Exceptions:**

  These are exceptions that are checked at compile-time by the Java compiler.

  Any method that might throw a checked exception must declare it using the throws keyword.

  Examples include IOException, SQLException, ClassNotFoundException, etc.

  Checked exceptions typically represent conditions that a well-behaved application should anticipate and recover from.

Un-Checked Exceptions

- **Un-Checked Exceptions:**

  These are exceptions that are not checked at compile-time; hence, the compiler does not force the caller to handle or declare them.

  They usually occur due to programming errors or conditions that are outside the programmer's control (e.g., null pointer dereference, arithmetic division by zero).

  Examples include NullPointerException, ArithmeticException, ArrayIndexOutOfBoundsException, etc.

Time for some ACTION

Airtribe

Airtribe Learner Manager

Associated session - session #1 (Introduction to java basics)

You need to develop a simple Learner Management System (LMS) for Airtribe. The system should manage learner records, including adding new learners, displaying learner details, and calculating the average XP of the learners. Your program should demonstrate your understanding of Java fundamentals covered in this week's session.

Main Program:

- Write a single Learner Management System class with the main method.
- Implement a menu-driven interface to:
  - Add new learners
  - Display all learner details
  - Calculate and display the average grade of all learners

Learner Management System

Control Statements:

Use if-else statements to validate learner age (should be between 18 and 100).

Use a switch statement to select actions from the menu.

Loops:

Use a loop to keep the menu active until the user chooses to exit.

Use loops to iterate through the list of learners to display details and calculate the average xp's.

Compilation and Execution:

Write a short note explaining the compilation and execution process.

Provide the commands used to compile and run the program.

Learner Management System

1. https://github.com/airtribe-projects/bel-c8-java

30-Min Live Exercise

Got Questions?

Airtribe

Thank you

Airtribe